

FinanceAI

PLAN ÚNICO DE CIERRE DEL MVP

Distribución precisa de trabajo: Fernando (Frontend) · Justin (Backend) · Jonathan (Integración + OCI)

Periodo de ejecución: lunes 17 a sábado 22 de agosto de 2026

Fecha límite no negociable

El sistema debe quedar funcional, probado, integrado y desplegado a más tardar el sábado 22 por la noche. La meta interna es tener toda la funcionalidad cerrada el viernes; el sábado se usa para corregir, documentar y ensayar.

Este documento reemplaza los planes anteriores

Se unifica aquí el estado revisado en GitHub, la incompatibilidad encontrada entre módulos, el trabajo de login/registro de Fernando, la seguridad ya iniciada por Justin, la base de datos, el histórico y el despliegue en OCI.

Resultado que debe poder demostrar el sábado

Registro → Login → Usuario autenticado → Perfil → Registrar/consultar transacciones → Análisis financiero → Comparar mes actual vs mes anterior → Ver recomendaciones → Todo funcionando desde OCI

1. Estado real del proyecto hoy

La distribución de trabajo se basa en el repositorio público revisado el 17 de agosto de 2026 y en las pruebas locales realizadas hoy.

Área	Ya existe	Falta cerrar	Dueño
Frontend	Dashboard de análisis; formulario; rama antigua con login/registro y dashboard adicional.	Consolidar rama; autenticación real; token en llamadas; datos personales; histórico; URL OCI; pruebas.	Fernando
Backend	Auth register/login; JWT; perfil; transacciones por usuario; health; análisis; Swagger; CORS local.	Cerrar seguridad; MySQL real; resumen/histórico; proteger análisis; pruebas de aislamiento; apoyar contrato.	Justin
Integración	Endpoint de análisis probado; frontend y backend funcionan por separado; módulo DS existe en rama antigua.	Unificar contrato; conectar Backend ↔ Data Science; fallback; integrar PRs; Docker.	Jonathan
OCI	Diseño y cuenta de OCI como objetivo del proyecto.	Registry; base MySQL; contenedor; variables/secretos; URL backend; publicar frontend; CORS producción; E2E.	Jonathan

Problema de rama que debe corregirse ya

En feature/integrar-frontend el frontend integrado contiene actualmente index.astro y gastos.astro. La rama frontend_fernando conserva trabajo adicional, incluyendo login/registro. Esa rama antigua se usará solo como fuente para recuperar archivos; ningún desarrollo nuevo debe continuar allí.

2. Lo que el Backend ya ofrece y no se debe rehacer

Endpoint	Uso	Estado
POST /api/v1/auth/register	Crear usuario	Existe
POST /api/v1/auth/login	Iniciar sesión y obtener token	Existe
GET /api/v1/users/profile	Consultar perfil del usuario autenticado	Existe
PUT /api/v1/users/profile	Actualizar perfil	Existe
POST /api/v1/transactions	Guardar una transacción del usuario	Existe
GET /api/v1/transactions	Consultar transacciones del usuario	Existe
GET /api/v1/health	Comprobar disponibilidad	Probado
POST /api/v1/analisis-financiero	Obtener análisis financiero	Probado; falta contrato final/protección
GET /api/v1/dashboard/summary	Resumen actual vs anterior	POR CREAR - Justin

3. Reglas de trabajo desde esta noche

1. feature/integrar-frontend será la rama común de integración.
2. Fernando y Justin crearán ramas nuevas a partir de feature/integrar-frontend. No deben seguir desarrollando en ramas antiguas.
3. Cada persona hará commit y push al terminar un bloque estable; luego abrirá Pull Request hacia feature/integrar-frontend.
4. Jonathan integra los PR solo después de que el responsable muestre sus pruebas.
5. No se agregan funciones nuevas después del viernes. El sábado es para bugs, documentación, OCI y demo.
6. Ninguna contraseña, DB_PASSWORD ni JWT_SECRET se sube a GitHub.
7. Swagger del Backend será la referencia oficial de los endpoints y formatos que consume Frontend.

Ramas recomendadas

```
feature/integrar-frontend
├── feature/frontend-final-fernando
├── feature/backend-final-justin
└── feature/oci-integration-jonathan
```

Regla de alcance

Si una función no es necesaria para demostrar el flujo Registro → Login → Datos → Análisis → Histórico → OCI, se pospone. Recuperar contraseña, login social, panel administrativo, roles avanzados y otros extras no son prioridad para esta semana.

4. FERNANDO - Frontend

Objetivo de Fernando

Entregar un frontend único, sin dependencia de su rama antigua, que permita registrarse, iniciar sesión, consultar y registrar datos propios, ver análisis e histórico, y funcionar con la URL pública de OCI.

4.1 Acciones inmediatas - esta noche / martes temprano

8. Hacer git fetch origin.
9. Cambiar a feature/integrar-frontend y actualizarla.
10. Crear feature/frontend-final-fernando.
11. Recuperar de frontend_fernando solamente lo necesario: login/registro, dashboard y componentes útiles. No copiar todo el repositorio antiguo encima del nuevo.
12. Confirmar que pnpm install, pnpm dev y pnpm build funcionan en la rama nueva.

4.2 Correcciones obligatorias de Frontend

- Eliminar el campo apellido del registro para el MVP, porque el Backend actual no lo recibe. No ampliar la base solo por ese dato esta semana.
- Usar POST /api/v1/auth/register y POST /api/v1/auth/login exactamente como los publique Swagger.
- Guardar el token después del login y usarlo en todas las llamadas privadas.
- Si no existe token, enviar al usuario a /login. Si existe token, permitir acceso al dashboard.
- Agregar cierre de sesión: borrar token/datos locales y volver a /login.
- Ocultar o desactivar “¿Olvidaste tu contraseña?” si no existe funcionalidad real de recuperación. No dejar un enlace que no hace nada.

- No dejar localhost:8080 escrito directamente como única URL. Usar una variable de configuración para que Jonathan pueda entregar la URL OCI sin modificar muchas pantallas.
- El dashboard debe mostrar datos del usuario autenticado, no valores de ejemplo.
- Consumir GET /api/v1/users/profile para el perfil.
- Consumir POST y GET /api/v1/transactions para guardar y listar transacciones reales.
- Consumir GET /api/v1/dashboard/summary cuando Justin lo entregue; Frontend solo dibuja la comparación, no inventa los cálculos.
- Consumir POST /api/v1/analisis-financiero usando el contrato final que publique Backend.
- Mostrar mensajes claros cuando el Backend esté caído, el token sea inválido o una solicitud falle.

4.3 Entregable de Fernando antes del viernes

Registro OK

Login OK

Dashboard protegido

Perfil real

Crear/listar transacciones

Análisis real

Mes actual vs mes anterior

Logout

API configurable

pnpm test / pnpm build OK

5. JUSTIN - Backend completo + Seguridad + Base de datos

Objetivo de Justin

Quitarle carga de Backend a Jonathan. Justin no se queda solo con SecurityConfig: debe dejar autenticación, seguridad, datos por usuario, MySQL, resumen/histórico y pruebas de Backend listos para que Jonathan pueda concentrarse en Data Science, Docker y OCI.

5.1 Seguridad - terminar primero

- Revisar JwtService, JwtAuthenticationFilter y SecurityConfig.
- Probar registro y login con contraseña correcta e incorrecta.
- Probar acceso sin token → 401.
- Probar token inválido → 401.
- Confirmar que perfil y transacciones requieren token.
- Actualmente /api/v1/analisis-financiero está público. Cambiarlo a protegido cuando Fernando ya envíe token correctamente.
- Mantener públicos únicamente health, auth y Swagger para la demo/documentación.
- Mantener CORS de localhost para desarrollo y preparar la configuración para aceptar la URL de Frontend OCI que entregará Jonathan.
- JWT_SECRET debe venir de variable de entorno, nunca quedar escrito como secreto real en Git.

5.2 Usuarios y transacciones - aprovechar lo que ya existe

- Validar GET/PUT /api/v1/users/profile con usuario autenticado.
- Validar POST /api/v1/transactions guarda la transacción con el usuario del token.
- Validar GET /api/v1/transactions devuelve solo datos de ese usuario.
- Prueba obligatoria con Usuario A y Usuario B: A nunca debe ver transacciones de B.
- Confirmar fecha, tipo, categoría, monto y descripción en las transacciones.

5.3 MySQL - responsabilidad de Justin en local

- Levantar MySQL local o usar una instancia de desarrollo acordada.
- Validar las migraciones Flyway existentes: V1 tabla inicial, V2 datos iniciales y V3 transacciones.
- Confirmar que DB_URL, DB_USERNAME y DB_PASSWORD funcionan desde variables de entorno.
- Entregar a Jonathan una lista de variables necesarias y el procedimiento de arranque, sin compartir contraseñas en Git.
- Probar que al reiniciar el Backend los usuarios y transacciones siguen existiendo en MySQL.

5.4 Histórico mínimo obligatorio

Para cumplir “histórico vs real” sin crear una arquitectura demasiado grande, el MVP compara el mes actual contra el mes anterior usando las transacciones que ya tienen fecha y usuario.

- Crear DashboardService y completar DashboardController.
- Crear GET /api/v1/dashboard/summary.
- Calcular para el usuario autenticado: ingresos del mes actual, gastos del mes actual, balance actual, ingresos del mes anterior, gastos del mes anterior, balance anterior y variación porcentual de gastos.
- Incluir totales por categoría del mes actual para la gráfica.
- No crear una tabla adicional de histórico si los cálculos pueden obtenerse de transacciones. Guardar análisis históricos queda como mejora posterior si sobra tiempo.

5.5 Entregable de Justin antes del viernes

Auth/JWT OK
401/seguridad OK
Perfil OK
Transacciones por usuario OK
MySQL persistente OK
Dashboard summary OK
Actual vs mes anterior OK
Prueba Usuario A $\neq$ Usuario B OK
Backend tests/compile OK
Swagger actualizado

6. JONATHAN - Integración, Data Science y OCI

Objetivo de Jonathan

No asumir todo el Backend. Jonathan define y prueba la unión entre módulos, integra los PR, conecta el análisis con Data Science y desde el jueves concentra la mayor parte de su tiempo en Docker y OCI.

6.1 Martes - contrato e integración Data Science

- Congelar el contrato oficial de POST /api/v1/analisis-financiero junto con Fernando y Justin.
- Backend es la fuente oficial: Frontend consume camelCase/API publicada; Backend traduce internamente si Python usa nombres distintos.
- Recuperar el módulo financeai-data-science existente de la rama antigua sin sobrescribir Backend/Frontend.
- Probar predict.py y modelos existentes. No reentrenar modelos salvo que sea indispensable.
- Crear/ajustar el servicio que invoca Data Science y devuelve una respuesta estable.
- Mantener AnalisisFinancieroService actual como fallback si el modelo falla, hasta que la integración quede estable.

6.2 Miércoles - cerrar integración local y empezar Docker

- Probar Frontend $\rightarrow$ Backend $\rightarrow$ Data Science de extremo a extremo.
- Integrar PR de Justin cuando pase seguridad/DB.
- Integrar PR de Fernando cuando login/dashboard compile y use token.
- Crear Dockerfile del Backend incluyendo lo necesario para ejecutar el módulo de Data Science.
- Probar la imagen en local antes de tocar OCI.

6.3 Jueves y viernes - OCI

13. Crear/validar VCN, subredes y reglas mínimas de red.
14. Crear repositorio en OCI Container Registry y subir la imagen del Backend + Data Science.
15. Crear MySQL HeatWave DB System en modo desarrollo/pruebas y obtener su endpoint privado/permitido.
16. Crear/configurar Container Instance para el Backend. OCI Container Instances permite ejecutar contenedores sin administrar servidores.
17. Configurar variables: DB_URL, DB_USERNAME, DB_PASSWORD, JWT_SECRET y cualquier variable necesaria para Data Science.
18. Arrancar Backend en OCI y validar GET /api/v1/health desde fuera.
19. Validar Swagger en OCI para tener evidencia de endpoints.
20. Entregar la URL pública del Backend a Fernando.

- 21. Publicar el frontend estático. Ruta preferida para el cierre: build de Astro y hosting estático en OCI Object Storage + API Gateway, si el tiempo lo permite; si esta ruta bloquea la entrega, usar un contenedor web simple como plan de contingencia.
- 22. Actualizar CORS del Backend con la URL pública final del Frontend.
- 23. Ejecutar prueba E2E pública: registro → login → transacción → análisis → histórico.

6.4 Entregable de Jonathan antes del sábado

Contrato final publicado
Backend  DS OK
PRs integrados
Docker local OK
OCI Registry OK
MySQL OCI OK
Backend OCI OK
Frontend OCI OK
CORS producción OK
E2E público OK
README/demo actualizados

7. Qué se guarda en la base de datos para el MVP

Dato	Se guarda	Uso
Usuario	Sí	Login, perfil y propiedad de datos
País / moneda	Sí	Preferencias del usuario
Transacciones	Sí	Histórico, comparación y análisis
Fecha de transacción	Sí	Mes actual vs mes anterior
Categoría / tipo / monto	Sí	Gráficas y resumen
Token JWT	No en BD para este MVP	Se usa para autenticar solicitudes
Contraseña en texto claro	Nunca	Debe almacenarse protegida/hasheada por Backend
Histórico de score de IA	No obligatorio esta semana	Puede agregarse después; el histórico mínimo sale de transacciones

Definición de histórico para esta entrega
“Histórico vs real” significa que el usuario autenticado puede comparar su situación del mes actual con el mes anterior usando los datos que él mismo registró. No se requiere construir un sistema histórico complejo de 12 meses para cerrar el MVP.

8. Contrato simple entre áreas

La regla es: Frontend no habla directamente con Data Science. Todo pasa por Backend.
Frontend → Backend (API oficial / Swagger) → Data Science → Backend → Frontend
Request de análisis a definir/cerrar como una sola versión. Ejemplo de dirección recomendada:

```
{
  "ingresoMensual": 4500,
  "nivelEndeudamiento": 25,
  "frecuenciaAhorro": "MEDIA",
  "moneda": "USD",
  "transacciones": [
    {"descripcion": "Compra supermercado", "valor": 420}
  ]
}
```

Backend puede traducir esos nombres al formato interno de Python. Fernando no debe consumir nombres internos como ingreso_mensual directamente si el contrato oficial del Backend usa otro formato.

9. Cronograma obligatorio: hoy a sábado

Día	Fernando	Justin	Jonathan
Lun 17 - noche	Crear rama final; identificar archivos a recuperar; no más cambios en frontend_fernando.	Crear rama final; actualizar Backend; compilar; listar pendientes de seguridad/DB.	Compartir este plan; congelar alcance; crear/ordenar tareas; no añadir nuevas funciones.
Mar 18	Integrar login/registro/dashboard; quitar apellido; token; logout; API configurable.	Cerrar JWT/401; perfil; transacciones; preparar MySQL; pruebas con token.	Cerrar contrato análisis; incorporar/probar Data Science; iniciar integración Backend ↔ DS.
Mié 19	Dashboard autenticado; listar/crear transacciones; consumir análisis.	DashboardController/Service; actual vs mes anterior; aislamiento A/B.	E2E local con DS; integrar PRs; Dockerfile.
Jue 20	Histórico visual; gráficas; errores; pnpm test/build.	MySQL persistente; Flyway; pruebas Backend; Swagger final.	Docker local; OCI Registry; VCN; MySQL OCI; iniciar Container Instance.
Vie 21	Cambiar a URL OCI; pruebas completas; solo corregir bugs.	Soporte a fallos de integración/seguridad/DB; no nuevas funciones.	Backend OCI; Frontend OCI; CORS producción; E2E público. FUNCIONALIDAD CONGELADA AL FINAL DEL DÍA.
Sáb 22	Pruebas finales en móvil/navegador; capturas; demo.	Pruebas finales auth/datos; apoyo bugs críticos.	Bugs críticos; documentación; README; evidencias; ensayo; entrega final por la noche.
Meta interna Viernes 21 por la noche: flujo funcional completo. Sábado 22: ninguna función nueva. Solo errores críticos, despliegue final, documentación, capturas y ensayo.			

10. Pruebas obligatorias antes de declarar terminado

Fernando - Frontend

- Registro válido.
- Registro con correo duplicado muestra error.
- Login correcto.
- Login incorrecto muestra error.
- Sin token no entra al dashboard.
- Con token sí entra.
- Perfil se carga desde Backend.
- Crear una transacción.
- Listar solo transacciones del usuario.
- Ver análisis y recomendaciones.
- Ver comparación mes actual vs anterior.
- Cerrar sesión.
- Backend caído muestra mensaje.
- pnpm test y pnpm build sin error.

Justin - Backend

- Compilación/tests sin error.
- Register 201.
- Login 200.
- Sin token 401 en endpoints privados.
- Token inválido 401.
- Perfil devuelve usuario correcto.
- Transacción se asocia al usuario del token.
- Usuario A no ve Usuario B.
- MySQL persiste tras reiniciar.
- Dashboard summary calcula períodos correctamente.
- Swagger refleja endpoints finales.
- CORS local y producción revisado.

Jonathan - Integración / OCI

- predict.py/modelos responden.
- Backend sigue respondiendo aunque DS tenga error controlado/fallback.
- Docker inicia sin depender de configuración local secreta.
- Backend OCI /health responde.
- Registro/login funciona contra OCI.
- DB OCI persiste datos.
- Frontend público consume Backend OCI.
- CORS correcto.
- Flujo completo desde una sesión nueva del navegador.
- README contiene instrucciones y URLs finales.

11. Checklist del sábado antes de entregar

Área	Criterio	OK
------	----------	----

Git	Todos los cambios están en feature/integrar-frontend y luego en la rama final acordada; working tree limpio.	☐
Frontend	URL pública abre; registro/login/dashboard/transacciones/histórico/análisis funcionan.	☐
Backend	Swagger abre; health UP; endpoints privados exigen token.	☐
BD	Usuarios/transacciones persisten; usuario A no ve datos B.	☐
Data Science	Análisis devuelve resultado útil o fallback controlado.	☐
OCI	Backend y frontend accesibles públicamente; secretos fuera de Git.	☐
Pruebas	Caso feliz + errores principales ejecutados.	☐
Documentación	README actualizado; arquitectura/flujo; variables necesarias; demo paso a paso.	☐
Demo	Datos de prueba preparados; secuencia ensayada; capturas de respaldo.	☐

Definición final de “terminado”

Una persona nueva puede abrir la URL pública, registrarse, iniciar sesión, registrar movimientos, consultar sus datos, ejecutar el análisis, comparar el mes actual con el anterior y cerrar sesión sin usar terminal, Postman ni editar archivos.

12. Fuentes usadas para este plan

Estado del código revisado en GitHub el 17 de agosto de 2026:

- Rama de integración: <https://github.com/No-Country-simulation/G9-LATAM-Team-77/tree/feature/integrar-frontend>
- Rama antigua de Frontend/DS: https://github.com/No-Country-simulation/G9-LATAM-Team-77/tree/frontend_fernando
- SecurityConfig: /Backend/financeia-backend/.../config/SecurityConfig.java
- AuthController, UserController, TransactionController y DashboardController en la rama de integración.
- Flyway: V1__create_tabla_inicial.sql, V2__insert_datos_iniciales.sql, V3__create_tabla_transacciones.sql.
- Login/registro de Fernando: financeai-frontend/src/pages/login.astro en frontend_fernando.

OCI (documentación oficial de Oracle consultada para el plan de despliegue):

- OCI Container Instances: <https://docs.oracle.com/en-us/iaas/Content/container-instances/overview-of-container-instances.htm>
- Creación de Container Instance / OCI Registry: <https://docs.oracle.com/en-us/iaas/Content/container-instances/creating-a-container-instance.htm>
- MySQL HeatWave DB System: <https://docs.oracle.com/iaas/mysql-database/doc/creating-db-system.html>
- Hosting estático con Object Storage + API Gateway: <https://docs.oracle.com/en/learn/oci-api-gateway-web-hosting/index.html>

Mensaje corto para el equipo

Compartir tal cual

Tenemos hasta el sábado por la noche. Fernando se queda dueño del Frontend final; Justin toma Backend completo, seguridad, MySQL e histórico; Jonathan se encarga de la integración con Data Science y, desde el jueves, OCI. La funcionalidad debe quedar cerrada el viernes. El sábado no se agregan funciones: se corrige, se documenta, se despliega y se ensaya.